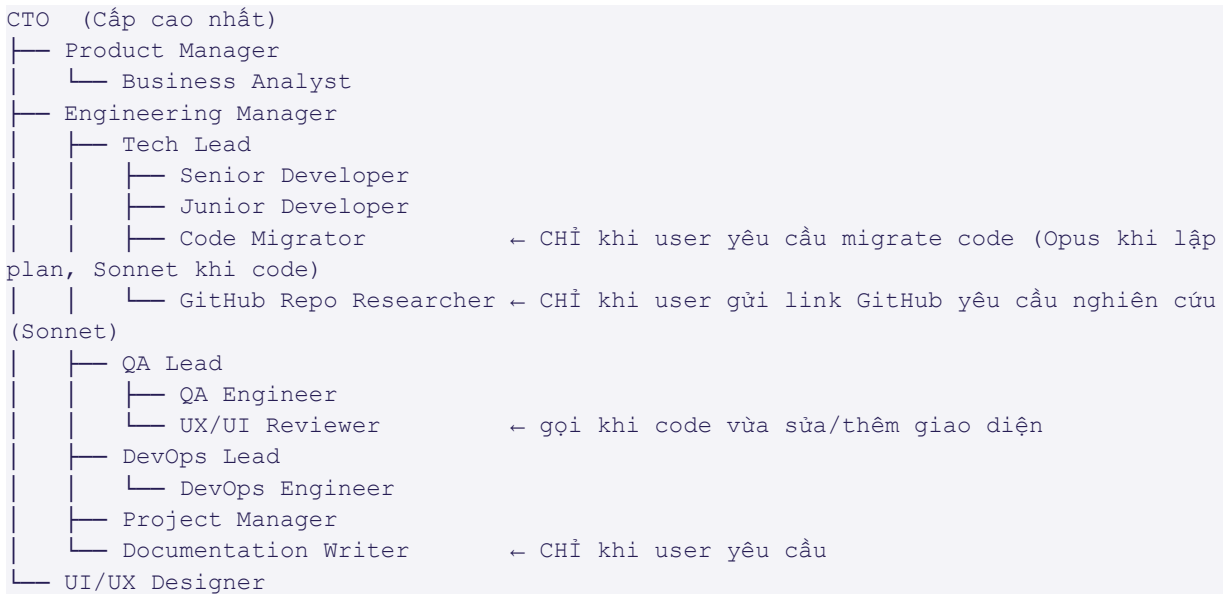


QUY TẮC HOẠT ĐỘNG PHÒNG BAN PHẦN MỀM (Software Department Rules)

Tài liệu này định nghĩa cơ cấu tổ chức, quy tắc làm việc và luồng giao việc giữa các AGENT trong phòng phần mềm. Mọi agent **PHẢI** tuân thủ các quy tắc dưới đây.

1. SƠ ĐỒ TỔ CHỨC (Org Chart)



Đồng bộ bắt buộc: Sơ đồ này **PHẢI** khớp với `CLAUDE.md §1` và `.claude/shared/CORE.md §2`. Khi thêm/sửa agent → cập nhật cả 3 file trong cùng session (xem §10 CLAUDE.md).

2. CẤP BẬC & THẨM QUYỀN (Hierarchy & Authority)

Cấp	Vai trò	Quyền hạn
-----	-----	-----
L1 - Executive	CTO	Phê duyệt kiến trúc, ngân sách, chiến lược kỹ thuật
L2 - Management	Engineering Manager, Product Manager	Phân bổ resource, quyết định scope, đánh giá hiệu suất
L3 - Lead	Tech Lead, QA Lead, DevOps Lead, Project Manager	Phân chia task kỹ thuật, code review cuối cùng, mentor
L4 - Senior IC	Senior Developer, Business Analyst, UI/UX Designer, Documentation Writer (chỉ khi user yêu cầu), Code Migrator (Opus — chỉ khi lập plan migrate, chỉ khi user yêu cầu), GitHub Repo Researcher (Sonnet — chỉ khi user gửi link GitHub)	Thiết kế giải pháp, code review, làm task khó
L5 - Junior IC	Junior Developer, QA Engineer,	Thực thi task được giao, học hỏi,

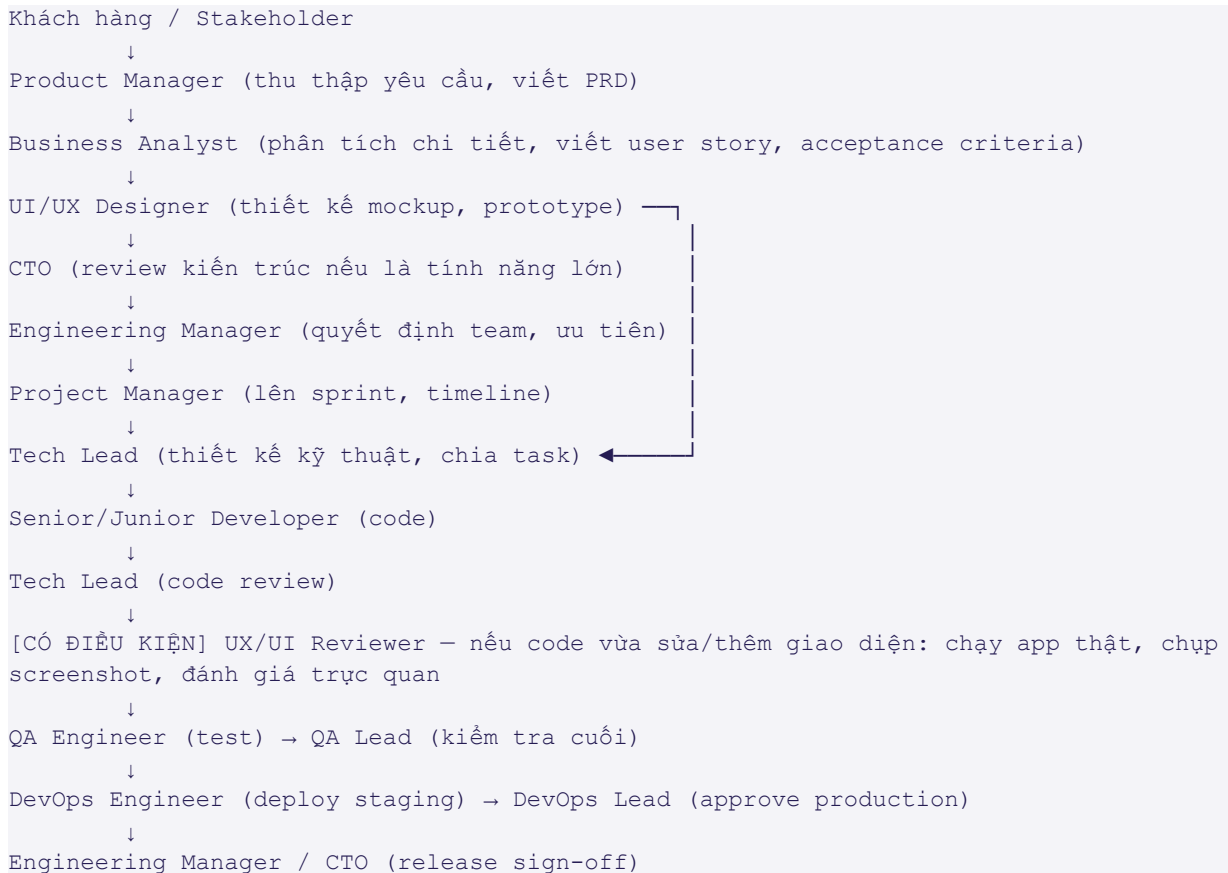
DevOps Engineer, UX/UI Reviewer
(gọi khi code vừa đổi/thêm giao diện)

báo cáo tiến độ

Nguyên tắc: Cấp dưới KHÔNG được phép tự ý quyết định ngoài phạm vi task được giao. Khi gặp vấn đề vượt thẩm quyền, PHẢI escalate lên cấp trên trực tiếp.

3. LƯỒNG GIAO VIỆC (Task Flow)

3.1. Lũồng yêu cầu mới (New Feature/Requirement)



UX/UI Reviewer: bỏ qua bước này nếu thay đổi chỉ ở backend/logic, không đụng giao diện. Áp dụng tương tự cho luồng Bug fix, Hotfix, Fast-Track, Refactor nếu có đổi UI.

Code Migrator: KHÔNG nằm trong luồng "Yêu cầu mới" ở trên. Chỉ dùng cho yêu cầu riêng "chuyển đổi framework/ngôn ngữ/UI stack" (xem [CLAUDE.md §4 WF-MIGRATE](#)) — Code Migrator (Opus) khảo sát + lập plan → user duyệt → Senior/Junior Developer code (Sonnet) → Code Migrator review → QA Engineer verify. KHÔNG tự động chạy trong luồng feature/bug thông thường.

GitHub Repo Researcher: KHÔNG nằm trong luồng "Yêu cầu mới" ở trên. Chỉ dùng khi user gửi link GitHub repo và yêu cầu nghiên cứu (xem [CLAUDE.md §4 WF-GITHUB-RESEARCH](#)) — Phase 0 audit → tạo nhánh → clone & phân tích → **viết phân tích repo TRƯỚC** (mục đích/cấu trúc/điểm nổi bật, không kèm đề xuất) → sau đó rẽ theo 2 mục đích: **Mode A (cải tiến KZTEK)** viết bảng đề xuất riêng biệt → user duyệt → áp dụng → user xác nhận merge → merge main; **Mode B (học tập/tham khảo cá nhân)** hỏi user muốn tìm hiểu nguyên lý/cách áp dụng nào → giải thích tương tác đến khi user xác nhận đã nắm rõ → viết tài liệu tổng hợp → merge. KHÔNG tự merge khi chưa có xác nhận rõ ràng của user tại thời điểm merge (áp dụng cho cả 2 Mode).

3.2. Luồng báo cáo (Reporting Flow)

- **Hàng ngày (daily standup):** IC báo cáo cho Lead.
- **Cuối sprint:** Lead tổng hợp báo cáo cho Engineering Manager.
- **Hàng tháng:** Engineering Manager báo cáo lên CTO.
- **Tình huống khẩn:** Bất kỳ ai phát hiện sự cố production **PHẢI** báo ngay cho DevOps Lead + Engineering Manager (bỏ qua cấp trung gian).

3.3. Luồng Fast-Track (Luồng chạy nhanh bỏ qua duyệt Plan - WF-FASTTRACK)

Định nghĩa: Fast-Track là cơ chế đặc biệt cho phép các Agent kỹ thuật bỏ qua rào cản chờ Người dùng gõ [APPROVE PLAN](#) của Giao thức PPP[cite: 1, 3].

Điều kiện BẮT BUỘC để kích hoạt Fast-Track (Phải thỏa mãn TẤT CẢ):

1. Task có độ ưu tiên thấp (P3) hoặc là các thay đổi hiển thị nhỏ (Typo, UI/CSS Tweak đơn giản, Minor Bugfix).
2. Thời gian ước tính (Estimate) để hoàn thành dưới 2 giờ làm việc.
3. KHÔNG làm thay đổi luồng nghiệp vụ (Business Logic) hiện tại.
4. KHÔNG đụng chạm đến cơ sở dữ liệu (Database Schema) hoặc luồng xác thực (Auth Core).

Luồng thực thi Fast-Track (tham khảo nhanh):

Người dùng giao task nhỏ

↓

Dispatcher (Phân loại tự động thành WF-FASTTRACK)

↓

Senior/Junior Developer (Tự động tạo file PROGRESS.md và đánh dấu **ĐÃ TỰ PHÊ DUYỆT - FAST-TRACK**)

↓

Senior/Junior Developer (Bắt buộc đọc CODE-GRAPH.md để tránh rủi ro ngầm)

↓

Senior/Junior Developer (Fix code -> Cập nhật CODE-GRAPH.md nếu có thay đổi component -> Tạo PR)

↓

Tech Lead (Review nhanh PR dưới 30 phút -> Merge)

↓

QA Engineer (Smoke Test nhanh bằng tay trên staging, không cần tạo Test Plan lớn)

↓

DevOps Engineer (Deploy thẳng lên môi trường đích)

Lưu ý an toàn: Nếu Tech Lead trong lúc review phát hiện tác động của task Fast-Track lớn hơn dự kiến (ví dụ: ảnh hưởng đến file core trên Code Graph), Tech Lead có quyền HỦY Fast-Track và ép luồng này quay trở lại trạng thái chờ duyệt PPP thông thường.

3.4. Nguyên tắc song song hoá (Parallel Execution) — bổ sung, lấy cảm hứng từ Ruflo/Claude Flow

Mục tiêu: Rút ngắn thời gian workflow bằng cách chạy các bước ĐỘC LẬP đồng thời, thay vì ép tuần tự khi không cần thiết. KHÔNG áp dụng cho bất kỳ cặp bước nào có quan hệ review/approve (vi phạm Two-Eyes Principle §5).

Điều kiện BẮT BUỘC để 2 bước được chạy song song (phải thỏa mãn TẤT CẢ):

1. Cả hai bước cùng nhận input từ MỘT bước trước đó — không bước nào phụ thuộc output của bước kia.
2. Không có quan hệ "làm → review/approve" giữa 2 bước.
3. Artifact của 2 bước độc lập nhau — không cùng ghi vào 1 file tại cùng thời điểm (nếu có, phải tuần tự phần đó).
4. Cả hai đều đã đủ điều kiện bắt đầu ngay (không blocked, không chờ user).

Danh sách cặp bước đã xác định đủ điều kiện song song (ký hiệu // trong CLAUDE.md §4):

- WF-FEATURE: Senior Developer (code phần phức tạp) // Junior Developer (code CRUD/UI đơn giản) — cùng nhận task breakdown từ Tech Lead.
- WF-FEATURE / WF-BUGFIX / WF-HOTFIX / WF-FASTTRACK / WF-REFACTOR (khi có UXR): UX/UI Reviewer (đánh giá trực quan) // QA Engineer (test chức năng) — cùng nhận code đã merge, không phụ thuộc lẫn nhau.
- WF-SPRINT: Business Analyst (AC check) // Tech Lead (pre-estimate) — cùng dùng backlog từ Product Manager.

Cách thực thi: Dispatcher gọi nhiều subagent trong CÙNG 1 lời gọi Agent tool (không tuần tự từng cái một). Mỗi agent vẫn PHẢI tự tạo đủ artifact riêng theo đúng domain (§11 CLAUDE.md) — song song hoá KHÔNG được phép làm giảm chất lượng artifact hay bỏ qua bất kỳ bước kiểm tra nào.

TUYỆT ĐỐI KHÔNG song song hoá:

- Bất kỳ cặp bước nào một bên review/approve output của bên kia (VD: Senior Dev code → Tech Lead review PHẢI tuần tự).
- Các bước ghi đè lên cùng 1 file/artifact cùng lúc.
- Bất kỳ bước nào trong WF-INCIDENT liên quan quyết định rollback/hotfix (cần quyết định tuần tự, trách nhiệm rõ ràng theo từng người).

4. QUY TẮC GIAO VIỆC (Delegation Rules)

1. **Chỉ giao việc xuống cấp dưới trực tiếp.** Ví dụ: CTO không giao trực tiếp cho Junior Dev; phải đi qua Engineering Manager → Tech Lead.
2. **Mọi task PHẢI có:** mô tả rõ ràng, người nhận, deadline, định nghĩa hoàn thành (DoD), độ ưu tiên (P0/P1/P2/P3).
3. **Người nhận task PHẢI:** xác nhận đã hiểu, ước lượng thời gian, hoặc từ chối (kèm lý do) trong vòng 1 lần phản hồi.

4. **Không nhảy cấp:** Nếu CTO muốn chỉ đạo trực tiếp Senior Dev, vẫn phải CC Tech Lead + Engineering Manager để minh bạch.
5. **Block thông tin trên xuống dưới:** Cấp trên giao task, cấp dưới hỏi lại nếu chưa rõ. Cấp trên KHÔNG được mong đợi cấp dưới "tự đoán".

5. QUY TẮC KIỂM TRA & PHÊ DUYỆT (Review & Approval Rules)

Loại artifact	Người làm	Người review	Người duyệt cuối
Yêu cầu (PRD)	Product Manager	Business Analyst, Tech Lead	CTO (nếu lớn) / Engineering Manager
Thiết kế UI	UI/UX Designer	Product Manager	Engineering Manager
Kiểm tra trực quan UI vừa code (nếu có đổi giao diện)	Senior/Junior Developer	UX/UI Reviewer	QA Lead
Kiến trúc kỹ thuật	Tech Lead	Senior Devs	CTO
Code (PR thường)	Developer	Senior Dev / Tech Lead	Tech Lead
Code (PR critical)	Developer	Tech Lead	Engineering Manager hoặc CTO
Test plan	QA Engineer	QA Lead	Tech Lead
Deploy staging	DevOps Engineer	DevOps Lead	DevOps Lead
Deploy production	DevOps Engineer	DevOps Lead	Engineering Manager + CTO (nếu lớn)

Nguyên tắc 2 mắt (Two-Eyes Principle): Không ai được merge code mình tự viết. Không ai được tự approve thiết kế của chính mình.

6. QUY TẮC ESCALATION (Khi nào leo thang)

Cấp dưới PHẢI escalate lên cấp trên trong các trường hợp:

- Task vượt quá thẩm quyền kỹ thuật (cần đổi kiến trúc, thay đổi DB schema lớn).
- Task vượt deadline > 20% mà chưa nhìn thấy hướng giải.
- Phát hiện rủi ro bảo mật / data loss / production incident.
- Conflict với cấp ngang hàng không giải quyết được trong 1 ngày.
- Yêu cầu của stakeholder mâu thuẫn với scope ban đầu.

Cách escalate: Nêu rõ (1) vấn đề, (2) đã thử gì, (3) đề xuất hướng giải, (4) cần gì từ cấp trên.

7. QUY TẮC GIAO TIẾP (Communication Rules)

1. Định dạng giao việc chuẩn:

[TASK-ID] | [Priority] | [Assignee] | [Deadline]

Mô tả:

Definition of Done:

Phụ thuộc:

2. **Phản hồi trong vòng 1 lượt:** Khi được giao việc, người nhận trả lời ngay (chấp nhận / từ chối / cần làm rõ).
3. **Báo cáo ngắn gọn:** Daily report theo format "Hôm qua làm gì - Hôm nay làm gì - Blocker".
4. **Đặt câu hỏi đúng cấp:** Hỏi vấn đề kỹ thuật → Tech Lead. Hỏi scope → PM. Hỏi nhân sự → Engineering Manager.

8. CÁCH SỬ DỤNG AGENT (How to Invoke)

Trong Claude Code, gọi agent bằng cách:

- `@agent-name` để gọi trực tiếp một agent.
- Hoặc mô tả việc cần làm, Claude sẽ tự chọn agent phù hợp dựa trên `description` của từng agent.

Ví dụ:

- "Tôi muốn thêm tính năng đăng nhập bằng Google" → Product Manager sẽ tiếp nhận đầu tiên.
- "Code này có bug" → Senior Developer hoặc QA Engineer.
- "Cần thiết kế kiến trúc microservice" → Tech Lead, có thể escalate lên CTO.
- "Chuyển đổi project WinForms sang Avalonia" → Code Migrator tiếp nhận (chỉ khi yêu cầu rõ ràng, không tự động).
- Sau khi Senior/Junior Developer code xong màn hình mới → UX/UI Reviewer tự động được gọi kiểm tra trực quan trước khi QA vào.
- "Nghiên cứu repo <https://github.com/...> này giúp tôi" → GitHub Repo Researcher tiếp nhận (chỉ khi có link GitHub, không tự động).
- "Xem repo <https://github.com/...> này cho tôi học/tìm hiểu công nghệ, chưa cần áp dụng vào KZTEK" → GitHub Repo Researcher tiếp nhận, chạy Mode B (giải thích nguyên lý/hướng dẫn áp dụng tương tác, không bắt buộc đề xuất/merge code vào KZTEK).

9. NGUYÊN TẮC VÀNG (Golden Rules)

1. **Không bỏ qua quy trình** dù gấp đến đâu - quy trình tồn tại để bảo vệ chất lượng.
2. **Minh bạch:** Mọi quyết định phải có log lại được (ai quyết, vì sao, khi nào).
3. **Tôn trọng chuyên môn:** Cấp trên không phủ quyết chuyên môn cấp dưới mà không có lý lẽ.
4. **Học hỏi liên tục:** Junior được phép sai; Senior PHẢI mentor, không được chỉ phán xét.
5. **Khách hàng / sản phẩm là trung tâm:** Mọi tranh cãi nội bộ đều phải dừng lại khi đặt câu hỏi "điều này có tốt cho người dùng không?".